INFO 7610 Topic Nat Lang Engr Meth/Tools

Natural Language Processing Basics and Tasks

Feifei Pan

Agenda

- Intro to Natural Language Processing
  - Some Recaps
- Natural Language Processing Tasks
  - Text classification
  - Name Entity Recognition
  - Summarization
  - Machine Translation
  - Question Answering
  - Advanced tasks and Emerging Topics

What is NLP

- **Natural Language Processing (NLP)** is a subfield of artificial intelligence (AI) and linguistics that focuses on enabling computers to understand, interpret, generate, and interact with human language in a meaningful way.
- **Key Aspects of NLP:**
  - **Understanding**: Parsing and interpreting text to derive meaning.
  - **Generation**: Producing natural language text, such as creating responses in chatbots.
  - **Interaction**: Facilitating human-computer communication in natural language, e.g., virtual assistants.

Why NLP is important

- NLP tasks play a crucial role in enabling machines to understand, interpret, and respond to human language. These tasks are at the core of applications ranging from chatbots and virtual assistants to advanced analytics and automatic translation systems.
- **Bridging Human-Machine Communication**
  - **Text Classification**: Categorizing emails as spam or not, detecting hate speech.
  - **Sentiment Analysis**: Analyzing customer feedback or social media sentiment.
- **Enhancing Accessibility**
  - **Speech-to-Text**: Transcribing spoken language into written text.
  - **Text-to-Speech**: Converting written text into spoken language.
  - **Machine Translation**: Translating text between languages (e.g., Google Translate).

Going Back to the Old Days

- Rule-based systems
  - **Advantages of Rule-Based Systems**
    1. **Explainability**: The rules are human-crafted and interpretable, making system behavior easy to understand.
    2. **Domain-Specific Customization**: Experts can design rules tailored to a specific domain (e.g., legal, medical).
    3. **Low Resource Requirements**: Rule-based systems can function without large datasets or training, unlike machine learning models.
  - **Challenges of Rule-Based Systems**
    1. **Scalability**: Writing and maintaining a vast set of rules for complex tasks is labor-intensive and time-consuming.
    2. **Ambiguity Handling**: Rule-based systems struggle with linguistic ambiguity (e.g., polysemy, homonyms) and nuanced context.
    3. **Lack of Generalization**: Systems often fail when faced with inputs that deviate from the predefined rules.
- Statistical Natural Language Processing
  - HMM, Naive Bayes, N-gram, CRF
- Neural Natural Language Processing
  - RNN, LSTM
- The magical Transformer
  - BERT, GPT, T5

N-Gram

- An **N-gram** is a sequence of N consecutive items (words, characters, etc.) from a given sample of text or speech. It is commonly used in Natural Language Processing (NLP) to analyze and predict sequences in language data.
- Bigram Example
  - "I love natural language processing. I love machine learning.”
    - "I love": 2 times
    - "love natural": 1 time
    - "natural language": 1 time
    - "language processing": 1 time
    - "love machine": 1 time
    - "machine learning": 1 time

Bi Gram Example

"I love natural language processing. I love machine learning."

- "I love": 2 times
- "love natural": 1 time
- "natural language": 1 time
- "language processing": 1 time
- "love machine": 1 time
- "machine learning": 1 time

The bigram probabilities are calculated using the formula:

$$P(w_i|w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

Example Calculations:

1. $P(\text{"love"}|\text{"I"})$:

$$P(\text{"love"}|\text{"I"}) = \frac{\text{Count("I love")}}{\text{Count("I")}} = \frac{2}{2} = 1.0$$

2. $P(\text{"natural"}|\text{"love"})$:

$$P(\text{"natural"}|\text{"love"}) = \frac{\text{Count("love natural")}}{\text{Count("love")}} = \frac{1}{2} = 0.5$$

Bi Gram Example

- Sentence Probability
  - “I love natural language processing,”

  P("I love natural language processing")=P("I")·P("love"|"I")·P("natural"|"love")·P("language"|"natural")·P("processing"|"language")
  - P(“I”) usually is set to 1
  - $P(\text{”love”}|\text{”I”}) = 1.0$
  - $P(\text{”natural”}|\text{”love”}) = 0.5$
  - $P(\text{”language”}|\text{”natural”}) = 1.0$
  - $P(\text{”processing”}|\text{”language”}) = 1.0$

$$P(\text{”I love natural language processing”}) = 1.0 \cdot 1.0 \cdot 0.5 \cdot 1.0 \cdot 1.0 = 0.5$$

Challenges in NLP: Ambiguity in Language

- Lexical ambiguity: Words and phrases with multiple meanings
  - Remember the bank example?
- Syntactic ambiguity: Multiple interpretations of sentence structure.
  - Syntactic ambiguity occurs when a sentence can be interpreted in multiple ways due to its structure.
    - Visiting relatives can be boring.
    - The chicken is ready to eat.
- Semantic ambiguity: Ambiguity in meaning due to context.

Challenges in NLP : Multilinguality and Low-Resource Languages

- Morphological complexity (e.g., Turkish or Finnish).
  - Morphological complexity refers to the richness and intricacy of word formation processes in a language, including how words are inflected, derived, or composed of smaller meaningful units (*morphemes*). Languages like Turkish and Finnish are known for their **agglutinative morphology**, where words are formed by combining numerous affixes, each with a specific grammatical or semantic function.
  - **Word Length**: Words in Turkish and Finnish can become very long and dense with information.
  - **Ambiguity**: Some affix combinations can lead to multiple interpretations, resolved by context.
  - **Learning Curve**: For non-native speakers, mastering the intricate affix rules and cases can be difficult.
- Lack of annotated data for low-resource languages.
- Code-switching (mixing languages in the same sentence).
  - **Code-switching** is the practice of alternating between two or more languages (or language varieties) within a single conversation, sentence, or phrase. It is a common phenomenon in multilingual communities and is influenced by cultural, social, and linguistic factors.

Challenges in NLP: Ethical Consideration

- Bias in datasets and models:
  - Gender, racial, or cultural biases in training data.
  - Amplification of societal stereotypes.
- Privacy concerns: Risks of using sensitive user data.
- Fairness and inclusivity in model development.

Modern Natural Language Processing Stack

- Typical Pipeline
  - Problem formulation
  - Data collection, processing and understanding
  - Tokenization
  - Embedding
  - Modeling
  - Evaluation
- Common toolkits and framework
  - NLTK, SpaCy
  - PyTorch, TensorFlow
  - Hugging Face

NLP Tasks

Foundation: Text Processing

- **Tokenization**:
  - Splitting text into smaller units, such as words, sentences, or subwords. Example: "Natural Language Processing" → ["Natural", "Language", "Processing"]
  - Word-Level Tokenization
  - Character-Level Tokenization
  - Sentence-Level Tokenization
- Examples
  - A word: hello
  - A subword or word-piece: inter, esting
  - A character: a, b, c
  - Even whitespace or punctuation: , .

Subword Tokenization Models

- **Subword-Level Tokenization**: breaks words down into smaller, meaningful sub-units, such as prefixes, suffixes, or even character-level tokens. → works perfect for unseen words
- **Byte Pair Encoding (BPE)**
  - BPE iteratively merges the most frequent pairs of characters or subword units in the text corpus.
  - It's commonly used in neural machine translation and language models.
  - Example: *low, lower, lowest*

BPE

- **Split Words into Characters**
  - Each word is split into individual characters, with a special boundary marker _ added to indicate word boundaries:
  - l o w, l o w e r, l o w e s t
- **Count All Adjacent Character Pairs**
  - Count the frequency of each pair of adjacent characters in the corpus
  - ('l', 'o'): 3, ('o', 'w'): 3, ('w', 'e'): 2, ('e', 'r'): 1, ('e', 's'): 1, ('s', 't'): 1
- **Merge the Most Frequent Pair**
  - Identify the most frequent pair and merge it into a single unit
  - The most frequent pair is ('l', 'o'), so merge it into lo
    - lo w, lo w e r, lo w e s t

BPE

- **Update the Pairs and Repeat**
  - Recalculate the frequencies of the updated pairs
  - ('lo', 'w'): 3, ('w', 'e'): 2, ('e', 'r'): 1, ('e', 's'): 1, ('s', 't'): 1
  - Merge the most frequent pair again:
  - The most frequent pair is ('lo', 'w'), so merge it into low
    - low, low e r, low e s t
- **Stop When Desired Vocabulary Size is Reached**
  - Continue the process of merging pairs until you reach a predefined number of merges or vocabulary size. For example: After further iterations, the final subword vocabulary might look like
    - ['low', 'e', 'r', 's', 't']

Subword Tokenization Models

- **WordPiece:** used in models like BERT, is similar to BPE but incorporates probabilities of subword token sequences to handle rare or out-of-vocabulary words more efficiently.
- **Example**:
  - Original text: *"unhappiness"*
  - Tokens: ["un", "##happiness"]
  - The "##" prefix indicates a subword, so "happiness" is recognized as a suffix to the prefix "un."

Subword Tokenization Models

- **SentencePiece:** an unsupervised text tokenizer and detokenizer that uses a model similar to BPE and WordPiece. It operates on raw text and can create subword-level tokens.
- **Example**:
  - Original text: *"language"*
  - Tokens: ["▁lan", "gu", "age"]
    (The "▁" indicates a word boundary.)
- **Challenges**:
  - Ambiguity in splitting rare or compound words.
  - Maintaining language-specific nuances.

Method	Use Cases
BPE	Transformer models like GPT-2, GPT-3, and OpenAI Codex.
SentencePiece	Multilingual models like T5, XLM, and models designed for languages without spaces (e.g., Chinese, Japanese).
WordPiece	BERT, RoBERTa, ALBERT, and similar models focusing on accuracy and semantic alignment.

Aspect	BPE	SentencePiece	WordPiece
Advantages	Simple and intuitive; works well for many languages.	Language-independent; handles raw text directly.	High-quality segmentation; integrates well with models like BERT.
Disadvantages	Sensitive to language preprocessing; may struggle with non-spaced languages.	Adds special tokens for spaces, which may impact interpretability.	Requires pre-tokenized input, less flexible for non-spaced languages.

Aspect	BPE	SentencePiece	WordPiece
Language Dependency	Language-specific preprocessing may be needed.	Language-agnostic (handles raw text directly).	Relies on pre-segmented words for input.
Subword Merging	Based purely on frequency of subword pairs.	Combines frequency-based (BPE) or probabilistic (Unigram) methods.	Uses probabilities to merge subwords.
Space as Token	Space may or may not be treated explicitly.	Spaces are explicitly marked with a special token.	Space is implicit in word-based tokenization.
Vocabulary Building	Based on frequency counts of character/subword pairs.	Optimized for likelihood or frequency, depending on the model.	Focuses on maximizing sentence likelihood.
OOV Handling	Breaks unknown words into subwords or characters.	Handles OOV words similarly by breaking into subwords.	Efficiently handles OOV by splitting into known subwords.
Application	Common in Transformer models like GPT-2.	Widely used in T5, XLM, and other language-agnostic models.	Standard in BERT and similar models.

Special Tokens

Models often use tokens like:

- <PAD>: padding
- <BOS>: beginning of sentence
- <EOS>: end of sentence
- <UNK>: unknown token
- <SEP>: separator

- These help control generation and training.

Deep Learning-Based Tokenization Models

- Some modern tokenization models are integrated into deep learning pipelines and can learn optimal tokenization schemes from data. These tokenizers work in conjunction with language models like BERT, GPT-3, and T5, and are designed to handle a variety of text inputs and languages without the need for manual rule-setting.
- **FastBPE**: A faster implementation of BPE designed for large-scale tokenization.
- **BERT Tokenizer**: Specifically designed for BERT, using WordPiece tokenization with a built-in vocabulary.

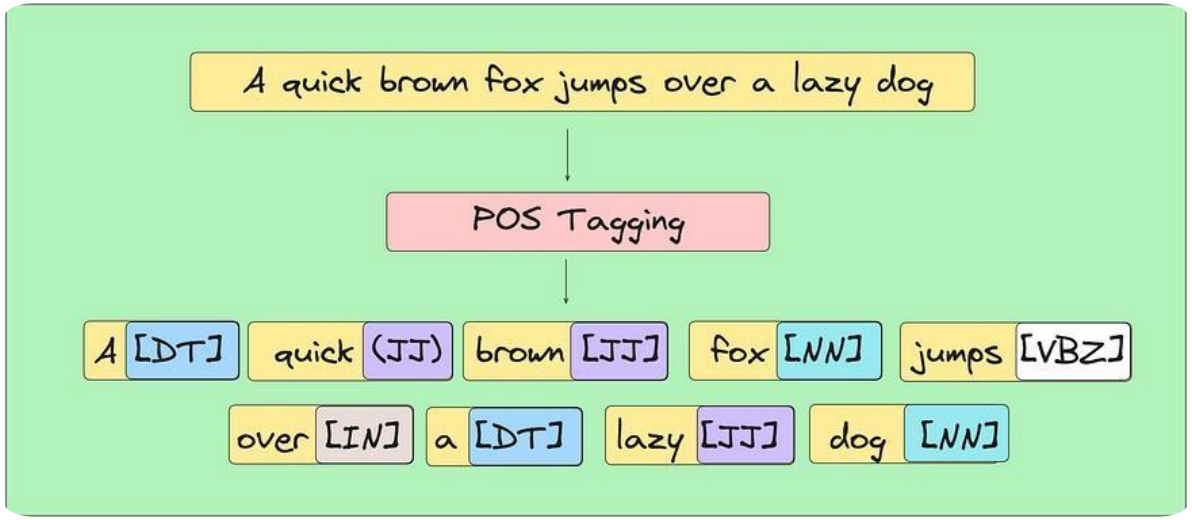
Choosing the Right Tokenization Model

- **Language**: Languages like Chinese, Japanese, and Korean require different tokenization approaches due to the absence of spaces between words.
- **Task**: For text generation, character-level tokenization may be suitable, whereas for classification or sentiment analysis, word-level or subword tokenization is more effective.
- **Data Availability**: Pretrained models like BERT and GPT may require specialized tokenization, but for smaller tasks, rule-based or simple tokenizers may suffice.

Foundation: Text Processing

- **Stemming and Lemmatization**:
  - Reducing words to their base form.
  - Stemming: Removing word suffixes (e.g., "running" → "run").
  - Lemmatization: Using vocabulary and grammar to find the root form (e.g., "better" → "good").
- **Text Normalization**: Converting text into a standard format
  - Lowercasing.
  - Removing punctuation, stop words, or special characters.
  - Expanding contractions (e.g., "don’t" → "do not").

Foundation: Text Processing



- **Part-of-Speech (POS) Tagging**
  - A task of assigning each word in a sentence with its corresponding grammatical category or part of speech (e.g., noun, verb, adjective, etc.). POS tagging is a crucial step in many NLP tasks, as it helps provide syntactic context to the words in a sentence, enabling better understanding of their relationships and roles in language.
  - Example: "*The cat sleeps*" → [The/DET, cat/NOUN, sleeps/VERB].
  - **Key POS Tags**
    - **Noun (NN)**: e.g., *dog, car, table*
    - **Verb (VB)**: e.g., *run, eat, sleep*
    - **Adjective (JJ)**: e.g., *beautiful, tall, fast*
    - **Adverb (RB)**: e.g., *quickly, slowly, well*
    - **Pronoun (PRP)**: e.g., *he, she, it, they*
    - **Preposition (IN)**: e.g., *in, on, at, under*

POS Tagging Methods

- Rule-Based POS Tagging
  - The system uses hand-crafted linguistic rules to assign tags to words based on their context. These rules can be based on word patterns, surrounding words, and the syntactic structure of the sentence.
- Statistical POS Tagging
  - Hidden Markov Models (HMMs):
    - HMMs are widely used for POS tagging. In an HMM, each state represents a POS tag, and the transitions between states are probabilistic, meaning the model predicts the most likely tag sequence based on observed word sequences.
  - Maximum Entropy Models:
    - Maximum entropy models are trained to maximize the entropy (uncertainty) of the output, subject to constraints learned from the training data. They work by considering the entire context in which a word appears, including surrounding words and their tags.

POS Tagging Methods

- Machine Learning-Based POS Tagging
  - Decision Trees: These models use a set of rules to classify words into different tags based on features. Decision trees split the data based on feature values (e.g., the presence of specific suffixes, word length, capitalization) to predict the most likely POS tag.
  - Support Vector Machines (SVMs): SVMs can be used to create a binary or multi-class classifier that assigns a POS tag based on various linguistic features of the words in a sentence.
  - Conditional Random Fields (CRFs): CRFs are a type of discriminative model that predict the sequence of POS tags based on contextual dependencies between words. CRFs are particularly effective for sequence labeling tasks like POS tagging, where the tag of one word depends on the tags of adjacent words.

POS Tagging Methods

- Deep Learning-Based POS Tagging
  - **Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) and Bidirectional LSTM (BiLSTM)**
    - RNNs can be trained to predict a sequence of tags based on the sequence of words.
  - **Transformer Models (e.g., BERT)**:
    - Transformer models like BERT (Bidirectional Encoder Representations from Transformers) have revolutionized NLP tasks, including POS tagging. They are based on self-attention mechanisms that allow the model to focus on all words in the sentence simultaneously and learn context-dependent representations of words.
    - **Example**: Fine-tuning BERT for POS tagging has proven highly effective because the model understands the full context of each word, making it robust to ambigu

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Ensure that the required NLTK datasets are downloaded
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

# Input sentence
sentence = input("Enter a sentence: ")

# Tokenize the sentence into words
tokens = word_tokenize(sentence)

# Perform POS tagging
tagged = pos_tag(tokens)

# Output the tagged words
print("\nPOS Tagging Output:")
for word, tag in tagged:
    print(f"{word}: {tag}")
```

Input Example:

```css
Enter a sentence: The quick brown fox jumps over the lazy dog.
```

Output Example:

```yaml
POS Tagging Output:
The: DT
quick: JJ
brown: NN
fox: NN
jumps: VBZ
over: IN
the: DT
lazy: JJ
dog: NN
```

POS Tagging in Practice

- NLTK

POS Tagging in Practice

- SpaCy

Input Example:

```css
Enter a sentence: The quick brown fox jumps over the lazy dog.
```

Output Example:

```yaml
POS Tagging Output:
The: DET
quick: ADJ
brown: ADJ
fox: NOUN
jumps: VERB
over: ADP
the: DET
lazy: ADJ
dog: NOUN
```

```python
import spacy

# Load the pre-trained SpaCy model
nlp = spacy.load("en_core_web_sm")

# Input sentence
sentence = input("Enter a sentence: ")

# Process the sentence using the SpaCy model
doc = nlp(sentence)

# Output the tagged words
print("\nPOS Tagging Output:")
for token in doc:
    print(f"{token.text}: {token.pos_}")
```

POS Tagging in Practice

- Stanford POS Tagger:
  - pip install stanfordnlp
- Transformer based methods:
  - https://huggingface.co/flair/pos-english

```
from stanfordnlp.server import CoreNLPClient

# Start Stanford NLP server (requires Java to be installed)
client = CoreNLPClient(annotators=['tokenize', 'ssplit', 'pos'], timeout=30000, me

# Input sentence
sentence = input("Enter a sentence: ")

# Annotate the sentence to get POS tagging
ann = client.annotate(sentence)

# Output the tagged words
print("\nPOS Tagging Output:")
for sentence in ann.sentence:
    for token in sentence.token:
        print(f"{token.word}: {token.pos}")

# Close the client when done
client.stop()
```

Challenges in POS Tagging

- **Ambiguity**:
  - Many words have multiple meanings and can function as different parts of speech depending on the context (e.g., *"lead"* can be a noun or a verb).
- **Out-of-Vocabulary Words**:
  - POS taggers often struggle with words that weren't present in the training data, especially for low-resource languages or new terminology.
- **Languages with Complex Morphology**:
  - Some languages (e.g., Turkish, Finnish) have complex morphological structures, making POS tagging more challenging due to the wide range of inflections and word forms.

Text Classification

- A task that involves assigning predefined categories or labels to a piece of text. It is one of the fundamental tasks in NLP and is used to organize, structure, and analyze text data.

Email	Spam Not Spam
Review	Positive Negative
Articles	Accounting Finance

Common Approaches to Text Classification

- **Rule-Based Methods**:
  - Use handcrafted rules like keyword matching or regular expressions.
  - Example: Emails with the word "WIN" in the subject line are marked as spam.
- **Machine Learning Models**:
  - Use algorithms like Naive Bayes, Support Vector Machines (SVM), or Logistic Regression.
  - Example: Train a model on labeled text data to classify movie reviews as positive or negative.
- **Deep Learning Models**:
  - Use neural networks like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), or Transformers (e.g., BERT, GPT).
  - Example: Fine-tune BERT for categorizing news articles.

Applications of Text Classification

- **Sentiment Analysis**: Analyzing opinions in reviews or social media.
- **Spam Detection**: Filtering out unwanted emails or messages.
- **Topic Classification**: Grouping news articles by topics like sports, politics, or entertainment.
- **Intent Recognition**: Identifying user intentions in chatbots (e.g., booking a flight or asking for the weather).
- **Language Detection**: Determining the language of a given text.
- **Document Categorization**: Organizing documents into categories for easier retrieval.

Named Entity Recognition (NER)

A natural language processing (NLP) technique that identifies and categorizes specific entities like people, places, organizations, dates, and quantities mentioned within a text

Types of Named Entities

- **Person**: e.g., *Albert Einstein, Barack Obama*
- **Organization**: e.g., *Google, Microsoft, United Nations*
- **Location**: e.g., *Paris, New York, Mount Everest*
- **Date/Time**: e.g., *January 20, 2025, yesterday, 10:00 AM*
- **Monetary values**: e.g., *$100, 500 euros*
- **Percentages**: e.g., *5%, 50% off*
- **Miscellaneous (e.g., event names, product names, etc.)**.

NER Approaches and Models

- **Rule-Based NER:**
  - rely on manually crafted rules and patterns for recognizing entities. These rules can be based on regular expressions, dictionaries of names, or syntactic patterns.
  - For example, a rule might state that any capitalized word following a specific pattern is a person's name.
- **Pros:**
  - Effective for specific domains with known named entities (e.g., medical entities).
  - Transparent and interpretable rules.
- **Cons:**
  - Inflexible and unable to handle unseen entities.
  - Time-consuming and difficult to scale or adapt to new data without manual intervention.

NER Approaches and Models

- **Statistical NER**
  - Statistical models use machine learning algorithms to identify named entities. These models are trained on labeled corpora, where each word in a sentence is annotated with its entity type.
  - **Hidden Markov Models (HMMs):**
    - HMMs are used to model sequential data and are based on probabilistic transitions between states (entity types). Each word is assigned a POS tag, and the transitions between words help predict the likelihood of a particular entity label.
  - **Conditional Random Fields (CRFs):**
    - CRFs are a type of discriminative model that assigns labels to sequences of words based on contextual features (e.g., surrounding words, capitalization, etc.). CRFs are widely used in NER for sequence labeling because they capture dependencies between adjacent tokens and are effective for handling ambiguities.

Statistical NER Model in Practice

- pip install hmmlearn
- Prepare the Training Data

```
X_train = [
    ['John', 'lives', 'in', 'New', 'York'],
    ['Jane', 'went', 'to', 'Paris'],
    ['Apple', 'is', 'based', 'in', 'Cupertino']
]

y_train = [
    ['PERSON', 'O', 'O', 'LOCATION', 'LOCATION'],
    ['PERSON', 'O', 'O', 'LOCATION'],
    ['ORGANIZATION', 'O', 'O', 'LOCATION']
]

# Prepare feature set and labels
words = [word for sentence in X_train for word in sentence]
labels = [label for sentence in y_train for label in sentence]
```

Statistical NER Model in Practice

- Encode words and labels--> build a model

```
# Encode words and labels
word_encoder = LabelEncoder()
word_encoder.fit(words)
X_train_encoded = [word_encoder.transform(sentence) for sentence in X_train]

label_encoder = LabelEncoder()
label_encoder.fit(labels)
y_train_encoded = [label_encoder.transform(sentence) for sentence in y_train]
```

```
# Create HMM model for NER
model = hmm.MultinomialHMM(n_components=len(label_encoder.classes_))

# Fit the model to the training data
for sentence, labels in zip(X_train_encoded, y_train_encoded):
    model.fit(sentence, lengths=[len(sentence)])
```

Statistical NER Model in Practice

Test the model on some samples…

```
# Predict named entities for a new sentence
test_sentence = ['John', 'moved', 'to', 'California']
test_sentence_encoded = word_encoder.transform(test_sentence).reshape(-1, 1)

# Predict using HMM
predicted_labels = model.predict(test_sentence_encoded)
predicted_tags = label_encoder.inverse_transform(predicted_labels)
```

Deep Learning-Based NER

- **Recurrent Neural Networks (RNNs)** process each word in a sequence one by one and maintain a "memory" of previous words.
  - LSTMs, and GRUs are more effective for NER tasks.
- **Bidirectional LSTMs (BiLSTMs)** process text in both directions (forward and backward) to capture context from both past and future words. For NER, BiLSTMs help capture the relationship between the words in a sentence and improve entity recognition accuracy.
- **Conditional Random Fields with BiLSTM (BiLSTM-CRF): BiLSTM-CRF** model combines the power of a BiLSTM for feature extraction with a CRF layer to label entities in a sequence. The CRF layer helps improve performance by considering the dependencies between adjacent tags
- **Transformers-Based NER (e.g., BERT, RoBERTa, GPT):** These models are pre-trained on large corpora and can be fine-tuned on NER datasets for highly accurate entity recognition. Transformers use a self-attention mechanism to model the relationships between all words in a sentence simultaneously, which leads to better handling of complex dependencies in text.
  - **Example: https://huggingface.co/dslim/bert-base-NER**

Applications of NER

- **Information Extraction**: Extracting key entities from unstructured data like news articles, web pages, and social media posts.
- **Question Answering**: Identifying entities relevant to a user's query, such as locations, dates, and people.
- **Knowledge Graphs**: Building structured knowledge representations by extracting and categorizing entities from large datasets.
- **Social Media Monitoring**: Detecting mentions of brand names, people, or organizations in social media posts.
- **Healthcare**: Extracting medical entities such as diseases, medications, or procedures from clinical texts.